Open a terminal and run the command *gcc:*

```
$ gcc
gcc: no input files
```

If you see something like the above output, *gcc* is already installed. If you see something like "Command not found", then you will have to install *gcc* using the package manager. Besides a compiler, you will also need the C standard library, called *glibc*, to compile your C programs correctly. Type in *locate glibc* and check the output. If it shows directory structures of the form '/foo/bar/*glibc*' or the like, then you have glibc installed; else you need to install it.

Okay, now that we have confirmed the presence of a text editor, a compiler and the standard library, let us write our first code in C on Linux. For the purpose of this article, let's create a sub-directory called 'codes' under the home directory, in which we will store all our source code.

Start up *gedit* and input this simple C code to print the factorial of a number:

```
#include<stdio.h>

int main(int argc, char **argv)
{
int n, i,fact=1;
printf("Enter a number for which you want to find the factorial:: ");
scanf("%d", &n);
for(i=1;i<=n;i++)
 fact=fact*i;

printf("Factorial of %d is :: %d\n", n,fact);
return 0;
}
```

Save this code in the *codes* sub-directory with the name 'fact.c' and use *cd codes* to go to this directory in your terminal. Once you are there, issue the following command:

```
$ gcc factorial.c
```

After executing the command, run *ls* and you will see an '*a.out*' file in the current directory. This is the executable file of your C program, compiled and linked with the appropriate libraries. To execute it, run (note the leading *./*, which is essential!):

```
$ ./a.out
Enter a number for which you want to find the factorial:: 5
Factorial of 5 is :: 120
```

Congratulations, you have just written your first C program on Linux! That was just the normal C that you write on DOS or Windows—no surprises there!

A bit more about this a.out file: This is the Linux equivalent of the *.exe* file that you would see under DOS/ Windows; it is the executable form of your code. As you might have already guessed, this file cannot be executed on DOS or Windows, since it is in a different format. Now, instead of having to rename your executable file each time you compile, you can specify the output file name to the compiler:

```
$ gcc -o factorial factorial.c
```

Try a few more programs from your C programming and data structures classes. 'The C Programming Language' is a well-known programming book by Brian Kernighan and Dennis Ritchie, which teaches you C programming with a strong Linux flavour. It would be a good idea to try the examples and exercise programs from this book to get a flavour of C programming on Linux.

Let's now write our first C++ program on Linux. The cycle of coding, compilation and execution is very similar to that for C, except for the compiler we use, which is g++. Check if it's already installed by running the command in a terminal, like we did for *gcc*. Next, use your package manager to check if *libstdc++*, the standard C++ library, is installed (if not, install it). Once both are installed, open up *gedit* and type this simple C++ program:

```
#include<iostream>
#include<string>

using namespace std;

int main(int argc, char **argv)
{
  string s1="Hello";
  string s2="World";

  cout <<s1+" " + s2 << "\n";

  return 0;
}
```

Save this file as *string-demo.cxx* in the *codes* sub-directory. Compile and execute the file:

```
$ g++ -o string-demo string-demo.cxx
$ ./string-demo
Hello World
```

The C++ code you see is standard C++, with the '.h' omitted from the header files. C++ source files conventionally use one of the suffixes '.C', '.cc', '.cpp', '.c++', '.cp', or '.cxx'.

Let us now write a simple C++ program that uses classes:

```
#include<iostream>
using namespace std;

class Circle{
```